AUFGABE 2.1 | SENSOR-PRODUCER

Wo steht welche Anforderung?

Lesehilfe zur vereinfachten IoT-Anwendung. Jede Fundstelle nennt Datei und Zeile im Projektordner. Stand: 26.10.2026. Die Verweise beziehen sich auf die aktuelle lokale Fassung; spätere Änderungen können Zeilennummern verschieben.

Der Nachrichtenfluss: producer.py sendet Messungen an RabbitMQ. Zwei Instanzen von analysis.py werten unterschiedliche Typen aus. alarm.py bearbeitet Alarme, monitor.py verarbeitet Kennzahlen. common.py enthält die gemeinsame RabbitMQ-Konfiguration.

Anforderung aus 2.1	Implementierung	Fundstelle
Mindestens drei Sensortypen	SENSORS definiert Temperatur, Luftfeuchtigkeit und Vibration mit Wertebereichen.	`producer.py:10`
Kontinuierliche Simulation	while True und eine Schleife über alle drei Typen erzeugen fortlaufend Zufallswerte; Pause: 0,5 s je Messung.	`producer.py:30-44`
Vier JSON-Pflichtfelder	sensor_id, timestamp, type und value entstehen hier. message_id ist eine zusätzliche Kennung.	`producer.py:35-37`
JSON und persistente Nachrichten	json.dumps serialisiert den Inhalt; delivery_mode=2 setzt Persistenz. Queues sind zusätzlich durable.	`common.py:64-69` `common.py:35`
Sinnvoller Routing Key	sensors.<type>.<sensor_id>, zum Beispiel sensors.temperature.temperature-01.	`common.py:72-74`
Verbindungsfehler und Retry	AMQP-/Netzwerkfehler lösen einen neuen Verbindungsaufbau und eine begrenzte Wiederholung derselben Messung aus.	`producer.py:13-27`

Warum range(6), obwohl nur fünf Wiederholungen erlaubt sind?

Es gibt einen ersten Sendeversuch und danach höchstens fünf Wiederholungen. Die Pausen betragen 1, 2, 4, 8 und 10 Sekunden. Damit wird die geforderte Obergrenze eingehalten.

```
14      for attempt in range(6):  # Ein Erstversuch und maximal fuenf Wiederholungen.
23              if attempt == 5:
24                  raise
25              delay = min(2 ** attempt, 10)
27              time.sleep(delay)
```

Zusätzliche Absicherung: common.py:57 aktiviert Publisher Confirms; common.py:67 setzt mandatory=True. Ein verlorenes Confirm kann zu einer erneuten Veröffentlichung derselben ID führen. Netzwerkfehler während der Pause werden in producer.py:40-44 erkannt.

AUFGABE 2.2 | ANALYSE-CONSUMER

Routing, Aggregation und ACK

analysis.py wird als zwei getrennte Prozesse gestartet: environment verarbeitet Temperatur und Luftfeuchtigkeit; machinery verarbeitet Vibration. Die Auswahl der Daten erfolgt bereits durch RabbitMQ-Bindings.

Anforderung aus 2.2	Was passiert?	Fundstelle
Mindestens zwei Analyse-Consumer	start.py startet analysis.py mit environment und machinery. Jede Instanz liest ihre eigene Queue.	`start.py:26-27` `analysis.py:25-33`
Filterung im Broker	Die Umgebungsqueue bindet temperature/humidity; die Maschinenqueue bindet vibration.	`common.py:36-40`
Gültige Messungen prüfen	JSON, Pflichtfelder, endliche Zahlen, Zeitstempel und Routing-Key-Konsistenz werden validiert.	`common.py:84-108`
10-Sekunden-Fenster	Monotone Uhr plus inactivity_timeout: Der Timer funktioniert auch ohne neue Nachricht. Nur Temperaturen kommen in pending.	`analysis.py:29-48` `analysis.py:68-70`
Aggregation ausgeben	Anzahl, Mittelwert, Minimum und Maximum werden berechnet, publiziert und ausgegeben.	`analysis.py:20-22` `analysis.py:37-39`
Separate Alarmnachrichten	Ein eigener Publish erzeugt die Alarmnachricht mit Stufe und AMQP-Priorität.	`analysis.py:64-66`
Manuelle ACKs/NACKs	auto_ack=False; erfolgreiche Lieferungen werden bewusst bestätigt. Ungültige Daten: NACK ohne Requeue.	`analysis.py:33` `analysis.py:59-74`

Anomaliebedingungen für alle drei Typen

Typ	WARNING	CRITICAL
Temperatur	> 80 °C	> 100 °C
Luftfeuchtigkeit	< 30 % oder > 70 %	< 15 % oder > 85 %
Vibration	> 5 mm/s	> 10 mm/s

Fundstelle: analysis.py:11-17. CRITICAL wird zuerst geprüft. Gleichheit mit einer Grenze erfüllt die jeweilige strikte Größer-/Kleiner-Bedingung noch nicht.

Temperaturen werden erst nach dem Fensterabschluss bestätigt

Die Werte bleiben mit ihren delivery_tags in pending. Erst wenn Aggregat und zugehörige Monitoring-Nachrichten erfolgreich publiziert wurden, folgt für jede Messung das ACK (analysis.py:37-45). Luftfeuchtigkeit und Vibration erhalten ihr ACK unmittelbar nach ihrer Verarbeitung und Telemetrie (analysis.py:71-75). So gehen bestätigte Temperaturen nicht schon vor der Aggregation allein im RAM verloren.

AUFGABEN 2.3 UND 2.4

Alarmpriorität, Exchanges und DLQ

2.3: Separater Alarm-Consumer

alarm.py:8-30 konsumiert ausschließlich die Alarmqueue. Eine gültige Alarmnachricht wird ausgegeben und danach manuell bestätigt (Zeilen 27-28). Fehlerhafte Alarme erhalten ein NACK mit requeue=False (Zeilen 22-25).

Baustein der Priorität	Implementierung	Fundstelle
Prioritätsfähige Queue	Classic-Queue mit x-max-priority=9.	`common.py:28-35`
Priorität pro Nachricht	WARNING erhält 3; CRITICAL erhält 9.	`analysis.py:64-66`
Wenig Vorabauslieferung	prefetch_count=1 lässt die Auswahl wartender Alarme beim Broker.	`alarm.py:10`

Die Bevorzugung gilt für wartende Nachrichten. Ein bereits laufender WARNING-Handler wird nicht durch einen späteren CRITICAL-Alarm unterbrochen.

2.4: Zwei unterschiedliche Exchange-Typen

Exchange	Typ und Routing	Zielqueue
iot.sensors	topic: sensors.temperature.* und sensors.humidity.*	iot.environment
iot.sensors	topic: sensors.vibration.*	iot.machinery
iot.events	direct: WARNING oder CRITICAL	iot.alarms
iot.events	direct: monitor	iot.monitor
iot.events	direct: dead	iot.dlq

Die gesamte Topologie wird in common.py:24-44 angelegt. Topic passt zu Kategorien und Sensor-IDs mit Wildcards. Direct genügt für die wenigen exakt benannten Ereignisse. Als Alternative würde Fanout Sensordaten an alle gebundenen Queues kopieren und zusätzliche Filterarbeit in den Analysen erfordern. Diese Begründung gehört in den Vortrag.

60 Sekunden TTL und der konkrete Weg zur DLQ

```
29          if queue != DLQ:
30              arguments.update({"x-message-ttl": 60000,
31                                "x-dead-letter-exchange": EVENT_EXCHANGE,
32                                "x-dead-letter-routing-key": "dead"})
```

Alle vier Hauptqueues bekommen diese Argumente; die DLQ ist ausgenommen. Ein NACK ohne Requeue oder eine abgelaufene wartende Nachricht führt über iot.events mit dem Schlüssel dead nach iot.dlq (common.py:44). Die DLQ hat keine TTL und keinen automatischen Rückweg.

TTL verhindert, dass beliebig alte Sensorwerte im normalen Rückstau verbleiben. Bei Ablauf lautet x-death.reason=expired. TTL ist ein Wartezeitlimit in der Queue, kein Verarbeitungstimeout für bereits ausgelieferte Nachrichten. Den tatsächlichen DLQ-Eintrag demonstriert demo.py:60-76.

AUFGABE 2.5 | MONITORING-CONSUMER

Kennzahlen kommen über RabbitMQ

Die Analysen greifen nicht auf Variablen im Monitoringprozess zu. Sie veröffentlichen eigene Ereignisse. monitor.py liest diese aus seiner RabbitMQ-Queue und berechnet daraus die geforderten Kennzahlen.

Geforderte Information	Variable / Berechnung	Fundstelle
Verarbeitete Messungen insgesamt	processed wird für status=processed erhöht.	`monitor.py:31-37`
Messungen je Sensortyp	per_type[kind] zählt Temperatur, Feuchtigkeit und Vibration getrennt.	`monitor.py:38`
Erkannte Anomalien / erzeugte Alarme	alarms zählt die von den Analysen übermittelte Alarm-Kennzeichnung.	`monitor.py:39`
Fehlgeschlagene Verarbeitungen	failures wird bei status=failed erhöht.	`monitor.py:41-42`
Mittlere Verarbeitungszeit	total_ms sammelt Zeiten; average = total_ms / processed.	`monitor.py:22-24` `monitor.py:40`

So gelangt eine Kennzahl zum Monitoring

Schritt	Code und Bedeutung
1. Ereignis erstellen	common.py:77-81: emit_metric enthält status, message_id, sensor_type, alarm, elapsed_ms und redelivered.
2. Über RabbitMQ publizieren	emit_metric ruft publish mit dem Routing Key monitor auf. common.py:43 bindet die Monitoringqueue daran.
3. Ereignis empfangen	monitor.py:20 liest MONITOR_QUEUE mit auto_ack=False; Zeile 29 dekodiert JSON.
4. Zählen und bestätigen	monitor.py:31-48 prüft die Daten, aktualisiert die Zähler und bestätigt danach mit ACK.
5. Regelmäßig ausgeben	monitor.py:17 und 21-25 berichten etwa alle fünf Sekunden, auch bei leerem Eingang.

Erzeugung von Erfolgs- und Fehlermeldungen

Fehlerhafte Sensornachrichten: analysis.py:59-62. Erfolgreiche Temperaturmessungen: analysis.py:40-43 nach dem Fensterabschluss. Erfolgreiche Feuchte-/Vibrationsmessungen: analysis.py:71-74. Auch ungültige Alarme melden einen Fehler über emit_metric (alarm.py:22-24).

Was die Zeitangabe genau bedeutet

analysis.py:53 startet die Messung mit time.perf_counter; Zeile 67 berechnet Millisekunden. Gemessen wird die lokale Bearbeitung bis zu diesem Punkt. Die Wartezeit im Broker und das spätere Warten auf den Temperatur-Fensterabschluss sind darin nicht enthalten.

Ein Aggregationsereignis wird ebenfalls übertragen. monitor.py akzeptiert status=aggregation, zählt es aber nicht als weitere Sensormessung (Zeilen 43-44). Die Zähler sind flüchtig; Neustarts, doppelte Ereignisse oder abgelaufene Telemetrie können den sichtbaren Stand verändern. Brokerseitig abgelaufene Messungen erzeugen selbst keine Failure-Telemetrie.

AUFGABE 2.6 | RESILIENZTESTS

Zwei gezielt reproduzierbare Fehler

Beide Szenarien benötigen einen laufenden RabbitMQ. demo.py erzeugt einen eigenen Namespace, startet eigene Analyseprozesse und kontrolliert die Beobachtungen. Die Befehle gelten im Projektordner:

```
.\.venv\Scripts\python.exe demo.py invalid
.\.venv\Scripts\python.exe demo.py crash
```

Fundstellen: ungültige Nachricht in demo.py:60-76; echter Prozessabbruch in demo.py:78-88. Die anschließende normale Verarbeitung wird in demo.py:89-91 nachgewiesen.

Frage aus der Aufgabe	invalid	crash
1. Was löst den Fehler aus?	Gezielt veröffentlichtes, syntaktisch ungültiges JSON.	Analyseprozess wird nach Empfang während einer Verzögerung von fünf Sekunden beendet.
2. Was passiert mit der Nachricht?	Sie erreicht die Umgebungsqueue; die Validierung schlägt fehl.	Sie ist zugestellt, aber noch unbestätigt. Nach dem Verbindungsabbruch wird sie erneut eingereiht.
3. ACK oder NACK?	NACK mit requeue=False; vorher wird Failure-Telemetrie publiziert.	Der beendete Prozess sendet weder ACK noch NACK. Der Ersatzconsumer bestätigt nach erfolgreicher Verarbeitung.
4. Erneute Zustellung?	Keine automatische erneute Verarbeitung in der Hauptqueue.	Ja: dieselbe ID crash-demo; die Telemetrie muss redelivered=True enthalten.
5. Wie greift die Behandlung?	Ungültige Daten werden dauerhaft ausgesondert. Ein unveränderter Wiederholungsversuch würde JSON nicht reparieren.	Der Broker hält unbestätigte Lieferungen verfügbar. Ein neu gestarteter Analyseprozess übernimmt die Messung.
6. Wann in die DLQ?	Sofort nach NACK ohne Requeue; die Demo liest den echten Eintrag und prüft reason=rejected.	Der Absturz allein erzeugt keinen Dead Letter. Spätere Ablehnung ohne Requeue oder Ablauf als wartende Nachricht kann dazu führen.
7. Läuft das System weiter?	Ja. Die gültige Folgenachricht after-demo wird verarbeitet.	Ja. Der Ersatzconsumer verarbeitet die erneut zugestellte Messung und die gültige Folgenachricht.

Die entscheidenden Code-Stellen

demo.py:62-64 sendet das kaputte JSON. Zeilen 66-73 lesen die tatsächliche DLQ-Nachricht und ihren x-death-Header. Beim Crash warten Zeilen 78-80 auf den Empfang; Zeile 82 beendet den Prozess. Zeilen 84-87 starten Ersatz und prüfen die Wiederzustellung.

Die Demo-Queues und -Exchanges werden am Ende entfernt (demo.py:92-103). Deshalb wird der DLQ-Beleg während der Demo ausgegeben und ist danach nicht als dauerhafter Eintrag in der Weboberfläche zu erwarten. Fehlerhafte Daten durchlaufen keine endlose requeue=True-Schleife.

ABSCHNITTE 3 BIS 5 | VORTRAG UND EINORDNUNG

Was ihr zusätzlich erklären müsst

Die Aufgabe bewertet die Gruppenpräsentation einschließlich Live-Demo. Neben dem Code zählen verständliche Entwurfsentscheidungen und die Erklärung des Verhaltens bei Fehlern. Die Vorbereitung liegt in docs/Praesentation.pdf und docs/VORTRAG.md.

Präsentationsteil	Passendes Material / Code
3.1 Architektur und Entscheidungen	docs/VORTRAG.md:9-17: zwei Exchanges, Queues und Alternative Fanout. Technische Umsetzung: common.py:24-44.
3.2 Zwei bis drei relevante Codeaspekte	Gute Auswahl: producer.py:13-27 (Backoff), analysis.py:35-45 (Fenster und ACK), common.py:29-34 plus analysis.py:59-61 (TTL/DLQ). Notizen: VORTRAG.md:25-45.
3.3 Live-Demo / Fehlerszenarien	docs/VORTRAG.md:47-56: Normalbetrieb, invalid und crash. Implementierung: demo.py:60-91.
3.4 Fazit	docs/VORTRAG.md:58-66: Erkenntnisse, Grenzen und sinnvolle Erweiterungen.

Der Vortragsplan umfasst 15 Minuten mit vier Minuten Live-Demo. Rollen und Gruppennamen ergänzt ihr selbst; alle Gruppenmitglieder müssen aktiv beteiligt sein. Die Präsentations-PDF wird entsprechend der Aufgabenstellung in Moodle abgegeben.

Die geforderten Technologien

Python ist die Programmiersprache aller Skripte. common.py:10 und 47-58 verwenden Pika für die AMQP-Verbindung zu RabbitMQ. requirements.txt:1 legt pika==1.3.2 fest. REST oder gRPC sind für diesen Nachrichtenfluss nicht erforderlich.

Empfohlene Reihenfolge zum Lesen des Codes

Reihenfolge	Was du dabei verstehen solltest
1. common.py:24	Welche Exchanges, Queues und Bindings gibt es?
2. producer.py:30	Wie entstehen Messwerte, und wie werden sie gesendet?
3. analysis.py:25	Wie wird validiert, analysiert, aggregiert und bestätigt?
4. alarm.py:8 / monitor.py:11	Wie werden Alarme und Kennzahlen getrennt verarbeitet?
5. demo.py:60 / start.py:26	Wie werden Fehler gezeigt und alle Dienste gestartet?

Grenzen, die ihr beim Erklären nennen solltet

- Ausgehender Alarm/Telemetrie und Quell-ACK sind nicht atomar. Mehrfachzustellung und doppelte Nebeneffekte sind möglich.
- Aggregation und Monitoring liegen im RAM; nach einem Ausfall können Werte in einem neuen Fenster erscheinen.
- Die Fehlerbehandlung sondert ungültige Daten aus. Für beliebig wiederholte echte Prozessabstürze gibt es keinen prozessübergreifenden Versuchszähler.
- Ein einzelner Broker bietet keine Hochverfügbarkeit bei Rechnerausfall.

Ergänzende Prüfungen: tests/test_project.py:46-97 für Fachregeln und AMQP-Einstellungen; Zeilen 21-43 prüfen die Konsolenausgabe des Starters; Zeilen 100-106 führen optional die beiden Broker-Demos aus.

Grundlage: API-Design-Bonusaufgabe-1-Message-Broker.pdf, Seiten 1-5. Codefundstellen beziehen sich auf C:/Coding/Python/Realtime-Analysis am 26.10.2026.